

Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria de Ingeniería Campus Zacatecas

Ingeniería en Mecatrónica

Sistemas Operativos en Tiempo Real

Reporte de Proyecto Final "Secadora de Filamento".

Docente:

Ramón Jaramillo Martínez.

Integrantes del equipo:

Andrea Bonilla Hernández.

Angelica Magdalena Hernández Acuña.

Jóse Espinoza Mora

Julio Alberto Nuñez Albarado

https://github.com/MagdaHdezAcuna/Sistemas_operativos_en_tiempo_real.git

Lunes, 12 De Enero De 2026

Resumen. – En el presente reporte presentamos los resultados del proyecto final de la materia de sistemas operativos en tiempo real... .

I. Introducción.

Una de las áreas de expansión más rápida de la explotación de computadores es aquella que implica aplicaciones cuya función principal no es el de procesar información, sino que precisa dicho proceso de información con el fin de realizar su función principal. Una lavadora controlada por microprocesador es un buen ejemplo de dicho tipo de sistemas. En este caso su función principal es la de lavar ropa; sin embargo, dependiendo del tipo de ropa que va a ser lavada, se deben ejecutar programas de lavado diferentes. Este tipo de aplicaciones de computador se conocen genéricamente como de tiempo real o embebidas.

Un sistema en tiempo real es cualquier sistema en el que el tiempo en el que se produce la salida es significativo. Generalmente es porque la entrada corresponde a algún movimiento en el mundo físico, y la salida está relacionada con dicho movimiento. El intervalo entre el tiempo de entrada y el de salida debe ser lo suficientemente pequeño para una temporalidad aceptable.

En el presente proyecto llevamos a la práctica lo aprendido y desarrollado durante el curso de sistemas operativos en tiempo real. El proyecto consiste en una secadora de filamento casera, en la cual se involucran elementos como el microcontrolador ESP32 el cual será el encargado de llevar a cabo los sistemas de control para lograr el objetivo que es mantener en una temperatura adecuada al filamento para secarlo. A través de este proyecto se llevó a cabo el uso de prioridades, semáforos, y el empleo de una base de datos. De la misma manera se integran estrategias de control PID y lógica difusa, lo cual nos permite determinar cómo un sistema operativo en tiempo real coordina algoritmos de control que requieren respuestas oportunas.

II. Antecedentes.

¿Qué es la impresión 3D?

Al proceso de creación de objetos por medio del depósito de capas de material unas sobre otras se le conoce como impresión 3D. La impresión 3D se denomina fabricación aditiva (AM) en lugar de los métodos sustractivos tradicionales, como el fresado CNC, cuando se utiliza para la producción industrial.

Esta tecnología existe desde hace cuatro décadas, fue inventada a principios de los 80, y aunque este método empezó siendo una técnica lenta y costosa, los amplios avances tecnológicos han hecho que las tecnologías AM actuales sean más asequibles y rápidas que nunca.

¿Cómo funciona la impresión 3D?

Básicamente es el corte de capas finas de un modelo digital que pasa a ser exportado en formato de código G, el cual es el lenguaje que la impresora 3D lee para saber con precisión cuándo y dónde depositar el material. De esta manera cada capa corresponde a la forma 2D exacta de una sección o rebanada del objeto. Estas capas se imprimen consecutivamente en 3D de una en una hasta obtener el objeto completamente impreso.

Tecnologías comunes de impresión 3D

Las principales tecnologías de impresión 3D presentes en la actualidad son:

- FFF (fabricación de filamento fundido) o FDM (modelado por deposición fundida), que utilizan bobinas de filamento
- SLA (estereolitografía), una tecnología que solidifica la resina fotosensible
- PBF (fusión de lecho de polvo), una serie de métodos basados en el polvo que fusionan partículas con potentes láseres
- El chorro de material o aglutinante, en el que se depositan diminutas gotas de material sobre un lecho de polvo [1]

Principales materiales de impresión 3D

Termoplásticos: Estándar, de ingeniería y de alto rendimiento

El plástico representa el material de impresión 3D más común. Podemos clasificar los termoplásticos de impresión 3D en tres tipos principales:

- Estándar
- De ingeniería o rendimiento
- De alto rendimiento

Materiales composite de impresión 3D

Los materiales composite pueden incluir elementos básicos como el PLA relleno de madera, pero también pueden contener materiales muy fuertes y resistentes como el Nylon relleno de fibra de carbono o el PEEK.

Metales y cerámica

La mayoría de estas soluciones son costosas máquinas de fabricación aditiva de grado industrial que producen piezas verdes. Antes de que estas piezas en bruto estén listas para su uso final, es necesario llevar a cabo pasos de posprocesamiento, como el descortezado o la sinterización.

Bioimpresión 3D

Los hidrogeles específicos suelen contener estas células. Pueden extruirse para hacer crecer piel o cartílago a bajas temperaturas.

¿Por qué los filamentos deben secarse?

La mayoría de los filamentos para impresión 3D son higroscópicos, lo que significa que absorben la humedad del ambiente. Esto ocurre especialmente en materiales como el PLA, ABS, Nylon y TPU. Si no se gestionan adecuadamente, estos filamentos pueden captar agua del aire, lo que afecta sus propiedades físicas y, por lo tanto, los resultados de impresión. Cuando un filamento contiene humedad, el agua atrapada se vaporiza durante el proceso de impresión, causando burbujas, hilos no deseados y una mala adhesión de las capas.

Secar los filamentos antes de usarlos es una de las mejores maneras de evitar estos problemas. La humedad no solo afecta la calidad estética de las impresiones, sino que también puede generar fallos en la extrusión y obstrucciones en la boquilla de la impresora. Esto puede causar una interrupción en el proceso de impresión, arruinando el proyecto o incluso dañando permanentemente el equipo.

Sistemas Operativos en Tiempo Real (RTOS)

Un sistema operativo en tiempo real (RTOS) es aquel sistema operativo especializado diseñado para gestionar tareas sensibles al tiempo con restricciones temporales precisas, así garantizando la previsibilidad y la estabilidad. Estos sistemas son importantes en aplicaciones de la automatización industrial, robótica, dispositivos médicos y sistemas integrados donde los retrasos o fallas pueden tener graves consecuencias. Los sistemas operativos en tiempo real también se utilizan comúnmente en entornos de alto riesgo donde las respuestas en tiempo real son esenciales para la seguridad y el rendimiento.[2]

Características clave de un sistema operativo en tiempo real

Características clave de un sistema operativo en tiempo real (RTOS):

- Determinismo: un RTOS garantiza que las tareas se ejecuten dentro de un marco de tiempo fijo y predecible, lo cual es esencial para las aplicaciones sensibles al tiempo.
- Multitarea: la multitarea es un RTOS implica programar y gestionar múltiples tareas, determinar el orden en que se ejecutan las tareas y cambiar entre ellas rápidamente, dando la impresión de ejecución paralela.
- Gestión de la memoria: un RTOS evita que las tareas interfieran con el espacio de memoria de las demás, mejorando la estabilidad y la seguridad del sistema.
- Latencia mínima: un RTOS reduce el tiempo de respuesta a eventos externos o interrupciones, lo que garantiza reacciones rápidas en entornos en tiempo real y minimiza la latencia de las interrupciones.
- Programación basada en prioridades: un RTOS ejecuta las tareas de mayor prioridad antes que las de menor prioridad, asegurándose de que las tareas críticas se manejen primero.
- Asignación de recursos: un RTOS maneja de manera eficiente la asignación de memoria, la potencia de procesamiento y otros recursos del sistema para respaldar el rendimiento en tiempo real.
- Manejo de interrupciones: un RTOS responde de manera rápida y eficiente a las interrupciones de hardware o software mediante mecanismos de interfaz de programación de aplicaciones (API) de RTOS.
- Sincronización de tareas: un RTOS proporciona comunicación entre tareas (ITC) con mecanismos como semáforos y colas de mensajes para sincronizar tareas y garantizar el intercambio seguro de recursos entre múltiples tareas.[2]

III. Desarrollo.

Para el secado del filamento se usó una resistencia PTC y para la disipación de aire caliente un ventilador plástico de 12V. Para llevar a cabo el control de estos sistemas, se implementó un controlador PID clásico y un PID con lógica difusa en cada sistema respectivamente. Las principales tareas que la secadora lleva a cabo es el control de temperatura por medio de los sistemas de la resistencia y el ventilador, el setpoint es modificable por medio de un pad numérico y el cambio se logra observar por medio de una pantalla lcd, además de la creación de una base de datos en donde se captura los datos obtenidos del sistema, para almacenarlos y de esta manera llevar a cabo el monitoreo en tiempo real.

Para ambos sistemas tanto el de la resistencia PTC como el ventilador, se diseñó una etapa de potencia. En la fig. 1 se muestra la etapa de potencia diseñada para el ventilador, esto debido a que el microcontrolador ESP32 maneja 3.3V, y el ventilador trabaja a 12V.

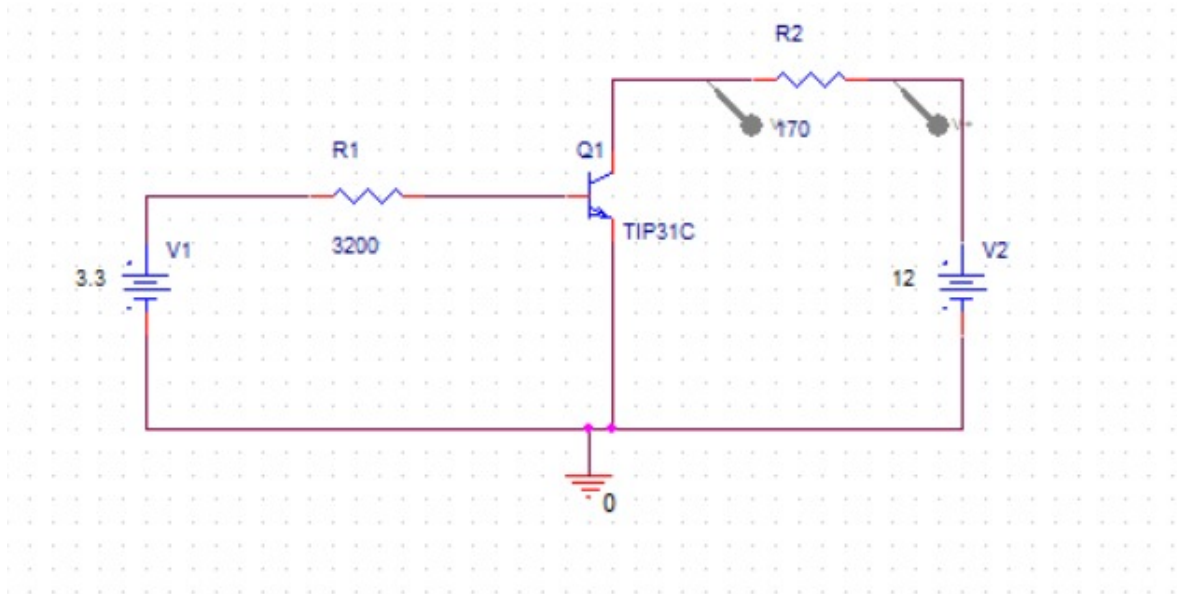


Figura 1. Etapa de potencia del ventilador

La resistencia PTC maneja corriente alterna y para poder controlar su temperatura térmica hicimos uso de un ssr, ya que de igual manera será controlado por el microcontrolador. También se diseñó una etapa de potencia para el ssr, ya que este se activa 3V DC, pero el pin de la ESP32 no garantiza que este trabaje, es por esto que se diseñó la etapa de potencia que se muestra en la fig. 2.

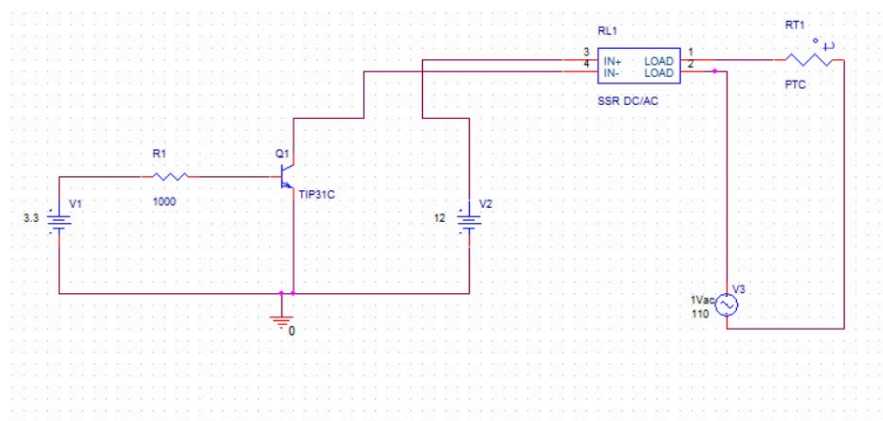


Figura 2. Etapa de potencia para el SSR.

La ventaja de usar RTOS en este tipo de proyectos, es la facilidad de ejecución simultánea de múltiples procesos, para este proyecto se realizaron tareas que se encargan de cumplir el propósito para la secadora de filamento:

TAREA 1: Lectura de sensores

Esta tarea se encarga de adquirir periódicamente la temperatura y la humedad del interior de la secadora de filamento utilizando el sensor digital SHT 31, y de almacenar estas mediciones de forma segura para que puedan ser utilizadas por las tareas de control, interfaz de usuario y monitoreo remoto.

En las siguientes líneas de código se puede apreciar los elementos de RTOS implementados.

```
// --- TAREA 1: Lectura de Sensores ---
void TaskSensors(void *pvParameters) {
    if (!sht31.begin(0x44)) {
        if (!sht31.begin(0x45)) {
            Serial.println("Error: No se encuentra SHT31.");
        }
    }

    while(1) {
        float t = sht31.readTemperature();
        float h = sht31.readHumidity();

        if (!isnan(t) && !isnan(h)) {
            if (xSemaphoreTake(xDataMutex, portMAX_DELAY)) {
                sharedTemp = t;
                sharedHum = h;
                xSemaphoreGive(xDataMutex);
            }
        }

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

TAREA 2: Control de actuadores

En esta tarea llevamos a cabo el control de los sistemas implementados, su función es procesar los datos de temperatura y humedad para generar las señales de control necesarias para que el calentador y el ventilador trabajen correctamente utilizando cada control respectivamente.

A grandes rasgos se lleva a cabo la inicialización de salidas de control, la configuración por ventana de tiempo, la temporización de la tarea, la lectura de las variables medidas.

Para el control del calentador configuramos el método de control, y su aplicación a la salida al SSR, es decir se convierte la salida del controlador en un tiempo de encendido dentro de la ventana, a mayor tiempo de activación del calentador mayor error térmico.

```
float heaterVal = 0;
                if (heater.useFuzzy) heaterVal =
computeFuzzyHeater(heater.setpoint - t);
    else heaterVal = computePID(heater, t, false);
    heater.output = heaterVal;
```

Para el control del ventilador, realizamos el mismo proceso, se implementa la selección del método de control según sea el caso, por lo tanto se configura distinto según sea el control elegido. Se realiza la sincronización temporal de la tarea evitando la acumulación de error temporal.

```
float fanVal = 0;
    if (fan.useFuzzy) fanVal = (h > fan.setpoint) ? map(h,
fan.setpoint, fan.setpoint+20, 60, 255) : 0;
    else fanVal = computePID(fan, h, true);
    fan.output = fanVal;
    ledcWrite(0, (int) fanVal);

    vTaskDelayUntil(&xLastWakeTime, xFrequency);
}
```

TAREA 3: Interfaz de usuario

Esta tarea se encarga de permitir la interacción directa del usuario con la secadora de filamento, esto para facilitar la visualización del sistema y la configuración dinámica de los

parámetros de control, como los setpoints, ganancias y el tipo de control. Como se utilizó una pantalla LCD, realizamos la inicialización de la interfaz, y definimos las variables locales de la tarea para el teclado matricial. De igual manera definimos la lectura del teclado matricial, y se definen las vistas en la pantalla, la primera de ellas la principal donde se muestran los setpoints, temperatura y humedad respectivamente. En la segunda vista aparece la selección del sistema para el control del calentador o el ventilador y en donde se usa el teclado. En la tercera vista se muestran las selecciones del parámetro, ganancias, setpoint y modo de control. Y por último una cuarta vista en donde se puede modificar la edición del valor. Estas visualizaciones se llevan a cabo como modos de estados como una máquina de estados.

TAREA 4: Sincronización firebase

El objetivo de esta tarea es enviar periódicamente el estado del sistema de la secadora a una base de datos en este caso Firebase, permitiendo el monitoreo remoto en tiempo real de las variables de proceso y de las señales de control, sin interferir con las tareas de control.

Como se ve en las siguientes líneas de código, se verifica la conexión a Wifi y mantiene un control para evitar bloqueos del sistema mientras se establece la conexión a internet.

```
void TaskFirebase(void *pvParameters) {  
    Serial.println("Esperando WiFi para Firebase...");  
    while (WiFi.status() != WL_CONNECTED)  
        vTaskDelay(pdMS_TO_TICKS(500));  
}
```

Para la configuración de Firebase, se asigna una URL de la base de datos, el token de autenticación y se define el tamaño de respuesta para evitar errores de memoria y se realiza la inicialización del cliente Firebase.

En el bucle de la tarea se muestra la preparación de los datos a enviar, para esto se creó un objeto JSON para empaquetar la información y se realiza la construcción del mismo.

De la misma manera se hizo uso de credenciales las cuales permiten que la ESP32 se autentique y acceda a la base de datos donde se almacenan y actualizan los datos del sistema.

A grandes rasgos la tarea de sincronización con Firebase se encarga de enviar periódicamente a la nube el estado del sistema, incluyendo las variables de proceso, los valores de referencia y las salidas de control.

En la Fig. 1 se muestra el contenedor donde se almacenarán los rollos de filamento donde se acoplaron los sistemas de secado tanto el ventilador como el calentador, el soporte donde se acoplaron los rollos y un respiradero de aire.

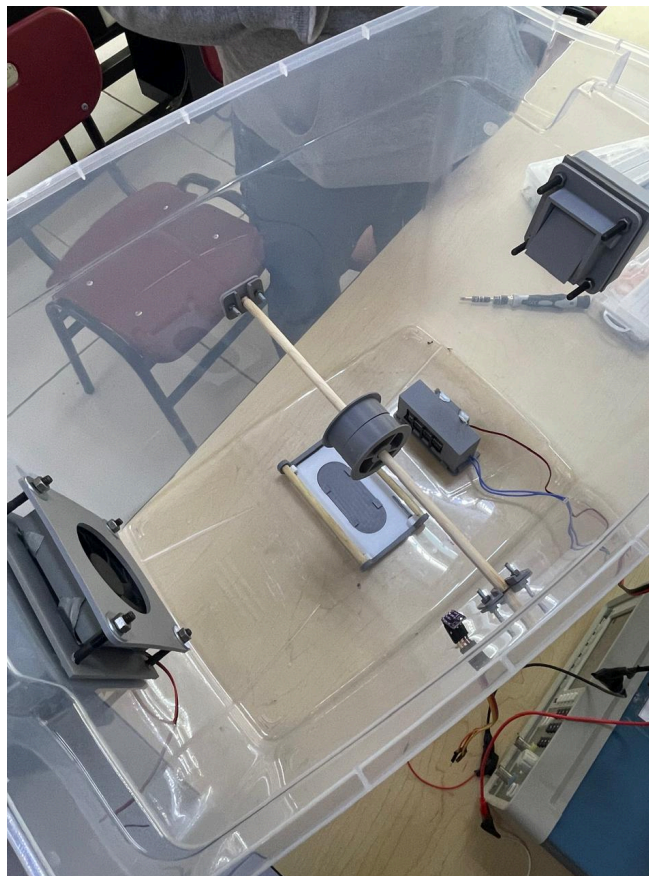


Figura 3. Secadora de filamento.

En la fig. 4 y 5 se muestra la interfaz del usuario y las etapas de potencia acopladas a la secadora de filamento, en la figura 4 se muestra la visualización de uno de los estados de la interfaz, esto ya en funcionamiento la secadora de filamento.

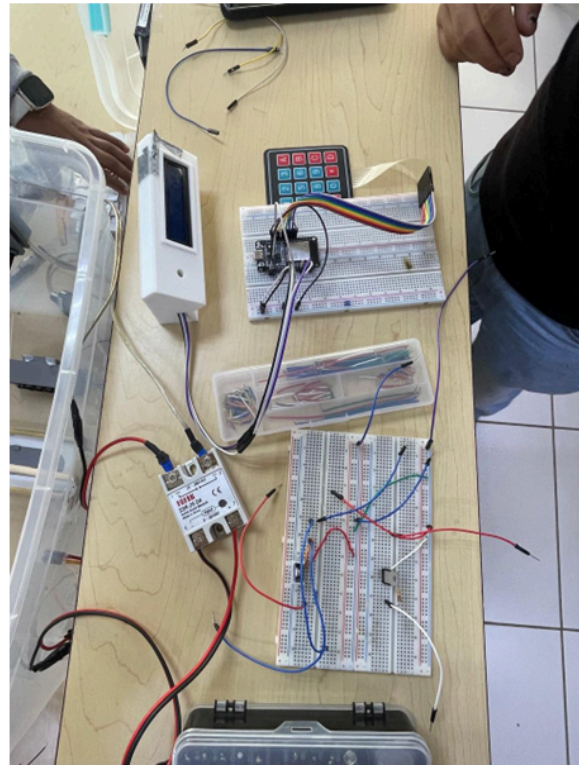
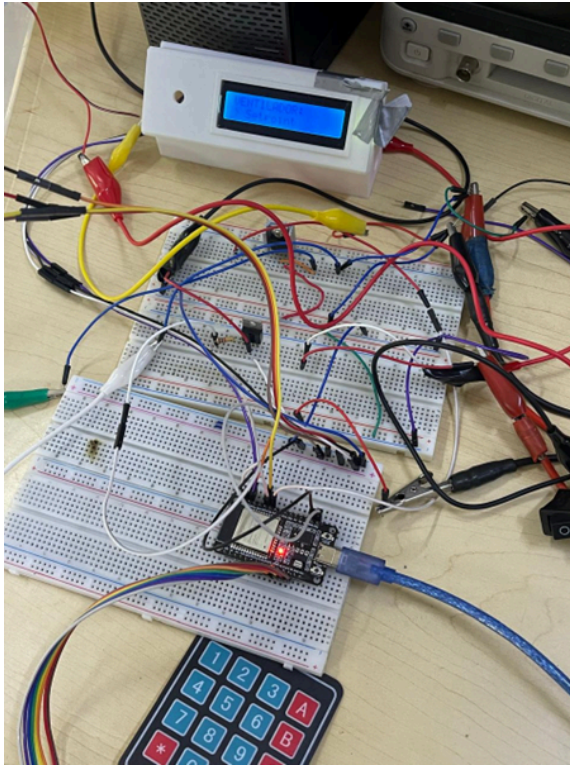
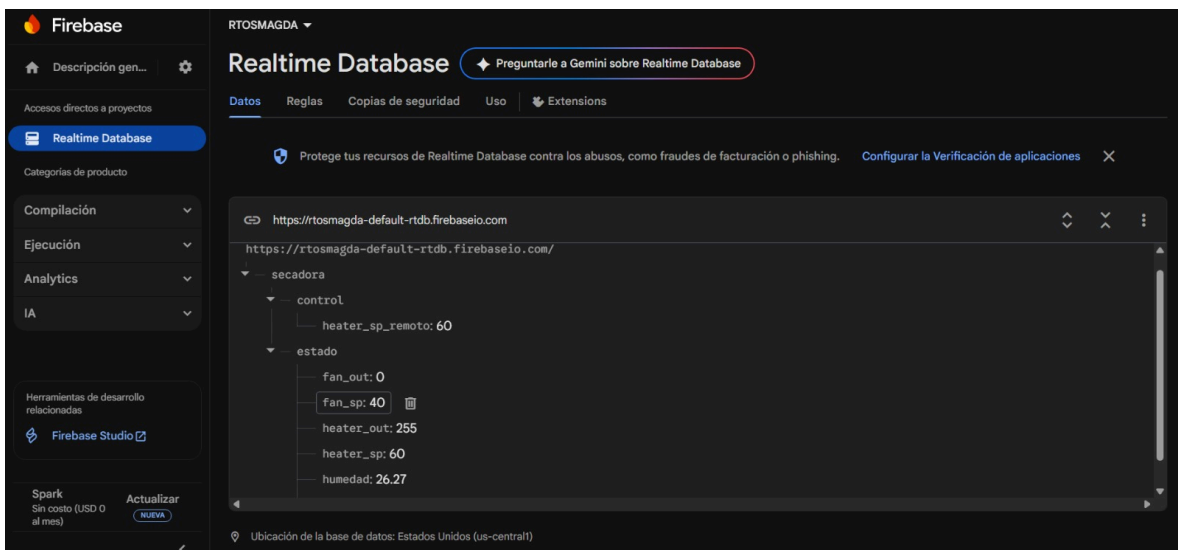


Figura 4 y 5. Interfaz de usuario y etapas de potencia.

En la fig. 6 se muestran los datos adquiridos del sistema en la base de datos creada en Firebase, en la cual se puede visualizar el estado del sistema en tiempo real es decir la temperatura, humedad, el setpoint de temperatura del calentador y la salida del controlador.



IV. Conclusión.

Este proyecto de una secadora de filamento, se integró un sistema operativo en tiempo real FreeRTOS, que se encarga de la gestión eficiente de múltiples tareas, en nuestro caso del control de los sistemas como el calentador y el ventilador, la interacción del usuario, la conexión con la base de datos y la lectura de los sensores. El uso de RTOS permitió mantener el código de una manera limpia y ordenada los procesos de adquisición de datos, el control de los actuadores, la interfaz y la comunicación con la nube, garantizando un comportamiento determinista y estable del sistema.

La incorporación de una interfaz de usuario mediante la pantalla LCD y el teclado matricial nos permite obtener una configuración dinámica mejorando su usabilidad. De igual manera el uso de la base de datos habilita el monitoreo remoto en tiempo real del estado de la secadora.

De forma general el presente proyecto nos permitió llevar a cabo la práctica las arquitecturas basadas en RTOS, el uso de las bases de datos logrando el objetivo de un sistema funcional, evidenciando que este tipo de implementación representa una solución eficiente para el secado de filamento.

V. Referencias bibliográficas.

[1] S. Susnjara and I. Smalley, "Sistema operativo en tiempo real (RTOS)," *Ibm.com*, Mar. 26, 2025. Available: <https://www.ibm.com/mx-es/think/topics/real-time-operating-system>. [Accessed: Jan. 08, 2026]

[1] "user," *Dassault Systèmes*, Jul. 05, 2022. Available: <https://www.3ds.com/es/make/guide/process/3d-printing>. [Accessed: Jan. 08, 2026]

VI. Anexos.

```
#include <Arduino.h>
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_SHT31.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
```

```

#include <FirebaseESP32.h>

/*===== credenciales =====*/

#define WIFI_SSID "MORONGUITAS"

#define WIFI_PASSWORD "delunoalnueve234"


#define FIREBASE_HOST "https://rtosmagda-default-rtdb.firebaseio.com/"

#define FIREBASE_AUTH "PGjWDtXFOFzWcA6hG47zAVqaplhYevNx4sYiarlr"


/*===== DEFINICIÓN DE PINES =====*/

#define PIN_HEATER      23  // Salida SSR (Slow PWM)

#define PIN_FAN         19  // Salida Ventilador (PWM Rápido)


// Pines del Keypad 4x4

#define R1 32

#define R2 33

#define R3 25

#define R4 26

#define C1 14

#define C2 27

#define C3 12

#define C4 13


Adafruit_SHT31 sht31 = Adafruit_SHT31();

LiquidCrystal_I2C lcd(0x27, 16, 2);

```

```

// Configuración Teclado

const byte ROWS = 4;

const byte COLS = 4;

char keys[ROWS][COLS] = {

    {'1','2','3','A'}, // A: Siguiente Sistema

    {'4','5','6','B'}, // B: Anterior Sistema

    {'7','8','9','C'}, // C: Entrar / Editar

    {'*','0','#','D'} // D: Atrás / Cancelar

};

byte rowPins[ROWS] = {R1, R2, R3, R4};

byte colPins[COLS] = {C1, C2, C3, C4};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS,
COLS);


// Objetos Firebase

FirebaseData fbData;

FirebaseAuth auth;

FirebaseConfig config;


struct ControlParams {

    float setpoint;

    float Kp;

    float Ki;

    float Kd;

```

```

    float output;

    float integral;

    float lastError;

    bool useFuzzy;
};

ControlParams heater = {50.0, 30.0, 0.1, 100.0, 0, 0, 0, false};
ControlParams fan     = {40.0, 5.0, 0.05, 10.0, 0, 0, 0, false};

float sharedTemp = 0.0;
float sharedHum  = 0.0;
SemaphoreHandle_t xDataMutex;

enum MenuState { HOME_VIEW, SELECT_SYSTEM, SELECT_PARAM, EDIT_VALUE };
MenuState currentMenu = HOME_VIEW;

int selectedSystem = 0;
int selectedParam  = 0;
float tempEditValue = 0;

/*=====LÓGICA DE CONTROL=====*/

float trimf(float x, float a, float b, float c) {

    if (x <= a || x >= c) return 0;

    if (x == b) return 1;

    if (x < b) return (x - a) / (b - a);

    return (c - x) / (c - b);
}

```



```

}

float computeFuzzyHeater(float error) {
    float eNeg = trimf(error, -50, -20, 0);
    float eZero = trimf(error, -5, 0, 5);
    float ePos = trimf(error, 0, 20, 50);
    float num = (eNeg * 0) + (eZero * 80) + (ePos * 255);
    float den = eNeg + eZero + ePos;
    if (den == 0) return (error > 0) ? 255 : 0;
    return constrain(num / den, 0, 255);
}

float computePID(ControlParams &sys, float input, bool reverseLogic) {
    float error = sys.setpoint - input;
    if (reverseLogic) error = input - sys.setpoint;
    sys.integral += error * 0.1;
    sys.integral = constrain(sys.integral, -100, 100);
    float derivative = error - sys.lastError;
    sys.lastError = error;
    float out = (sys.Kp * error) + (sys.Ki * sys.integral) + (sys.Kd *
derivative);
    return constrain(out, 0, 255);
}

/*=====TAREAS FREERTOS=====*/

```



```

// --- TAREA 1: Lectura de Sensores ---

void TaskSensors(void *pvParameters) {

    if (!sht31.begin(0x44)) {

        if (!sht31.begin(0x45)) {

            Serial.println("Error: No se encuentra SHT31.");

        }

    }

    while(1) {

        float t = sht31.readTemperature();

        float h = sht31.readHumidity();

        if (!isnan(t) && !isnan(h)) {

            if (xSemaphoreTake(xDataMutex, portMAX_DELAY)) {

                sharedTemp = t;

                sharedHum = h;

                xSemaphoreGive(xDataMutex);

            }

        }

        vTaskDelay(pdMS_TO_TICKS(1000));

    }

}

// --- TAREA 2: Control de Actuadores ---

void TaskControl(void *pvParameters) {

    ledcSetup(0, 20000, 8);

```

```

    ledcAttachPin(PIN_FAN, 0);

    pinMode(PIN_HEATER, OUTPUT);

    digitalWrite(PIN_HEATER, LOW);

    unsigned long windowStartTime = millis();

    const int WindowSize = 1000;

    TickType_t xLastWakeTime;

    const TickType_t xFrequency = pdMS_TO_TICKS(50);

    xLastWakeTime = xTaskGetTickCount();

    while(1) {

        float t, h;

        if (xSemaphoreTake(xDataMutex, portMAX_DELAY)) {

            t = sharedTemp;

            h = sharedHum;

            xSemaphoreGive(xDataMutex);

        }

        // HEATER

        float heaterVal = 0;

        if (heater.useFuzzy) heaterVal =
computeFuzzyHeater(heater.setpoint - t);

        else heaterVal = computePID(heater, t, false);

        heater.output = heaterVal;

        // SSR Logic

```

```

        long onTime = (long)((heaterVal / 255.0) * WindowSize);

        unsigned long now = millis();

        if (now - windowStartTime > WindowSize) windowStartTime +=
WindowSize;

        if ((now - windowStartTime) < onTime) digitalWrite(PIN_HEATER,
HIGH);

        else digitalWrite(PIN_HEATER, LOW);

// FAN

float fanVal = 0;

if (fan.useFuzzy) fanVal = (h > fan.setpoint) ? map(h,
fan.setpoint, fan.setpoint+20, 60, 255) : 0;

else fanVal = computePID(fan, h, true);

fan.output = fanVal;

ledcWrite(0, (int)fanVal);

vTaskDelayUntil(&xLastWakeTime, xFrequency);

}

}

// --- TAREA 3: Interfaz de Usuario ---

void TaskUI(void *pvParameters) {

    lcd.init();

    lcd.backlight();

    char key;

    unsigned long lastUpdate = 0;

```

```

while(1) {

    key = keypad.getKey();

    switch(currentMenu) {

        case HOME_VIEW:

            if (millis() - lastUpdate > 800) {

                lcd.setCursor(0,0); lcd.printf("T:%.1f H:%.1f",
sharedTemp, sharedHum);

                lcd.setCursor(0,1); lcd.printf("SP_T:%.0f
SP_H:%.0f", heater.setpoint, fan.setpoint);

                lastUpdate = millis();

            }

            if (key == 'C') { currentMenu = SELECT_SYSTEM;
lcd.clear(); }

            break;

        case SELECT_SYSTEM:

            lcd.setCursor(0,0); lcd.print("CONFIGURAR:");

            lcd.setCursor(0,1); lcd.print(selectedSystem == 0 ? ">
CALENTADOR" : "> VENTILADOR");

            if (key == 'A' || key == 'B') selectedSystem =
!selectedSystem;

            if (key == 'C') { currentMenu = SELECT_PARAM;
selectedParam = 0; lcd.clear(); }

            if (key == 'D') currentMenu = HOME_VIEW;

            break;

        case SELECT_PARAM:

            lcd.setCursor(0,0); lcd.print(selectedSystem==0 ?
"CALENTADOR:" : "VENTILADOR:");

            lcd.setCursor(0,1);

```

```

        switch(selectedParam) {

            case 0: lcd.print("> Setpoint"); break;

            case 1: lcd.print("> Kp"); break;

            case 2: lcd.print("> Ki"); break;

            case 3: lcd.print("> Kd"); break;

            case 4: lcd.print("> Modo Control"); break;

        }

        if (key == 'A') selectedParam = (selectedParam + 1) %
5;

        if (key == 'B') selectedParam = (selectedParam - 1 <
0) ? 4 : selectedParam - 1;

        if (key == 'D') currentMenu = SELECT_SYSTEM;

        if (key == 'C') {

            currentMenu = EDIT_VALUE;

            xSemaphoreTake(xDataMutex, portMAX_DELAY);

            ControlParams *sys = (selectedSystem==0) ? &heater
: &fan;

            if(selectedParam == 0) tempEditValue =
sys->setpoint;

            else if(selectedParam == 1) tempEditValue =
sys->Kp;

            else if(selectedParam == 2) tempEditValue =
sys->Ki;

            else if(selectedParam == 3) tempEditValue =
sys->Kd;

            else if(selectedParam == 4) tempEditValue =
(float) sys->useFuzzy;

            xSemaphoreGive(xDataMutex);

            lcd.clear();

```

```

    }

    break;

    case EDIT_VALUE:

        lcd.setCursor(0,0); lcd.print("Valor Actual:");

        lcd.setCursor(0,1);

        if (selectedParam == 4) lcd.print(tempEditValue > 0.5
? "FUZZY" : "PID CLASICO");

        else lcd.print(tempEditValue);

        float step = (selectedParam == 0) ? 1.0 : 0.1;

        if (key == 'A') tempEditValue += step;

        if (key == 'B') tempEditValue -= step;

        if (selectedParam == 4 && (key == 'A' || key == 'B'))
tempEditValue = !((bool)tempEditValue);

        if (key == 'D') currentMenu = SELECT_PARAM;

        if (key == 'C') {

            xSemaphoreTake(xDataMutex, portMAX_DELAY);

            ControlParams *sys = (selectedSystem==0) ? &heater
: &fan;

            if(selectedParam == 0) sys->setpoint =
tempEditValue;

            else if(selectedParam == 1) sys->Kp =
tempEditValue;

            else if(selectedParam == 2) sys->Ki =
tempEditValue;

            else if(selectedParam == 3) sys->Kd =
tempEditValue;

            else if(selectedParam == 4) sys->useFuzzy =
(bool)tempEditValue;

            xSemaphoreGive(xDataMutex);

```

```

        currentMenu = SELECT_PARAM;

        lcd.clear(); lcd.print("Guardado!");
vTaskDelay(500);

    }

    break;

}

vTaskDelay(pdMS_TO_TICKS(50));

}

}

// --- TAREA 4: Sincronización Firebase

void TaskFirebase(void *pvParameters) {

    Serial.println("Esperando WiFi para Firebase...");

    while (WiFi.status() != WL_CONNECTED)
vTaskDelay(pdMS_TO_TICKS(500));

    config.database_url = FIREBASE_HOST;

    config.signer.tokens.legacy_token = FIREBASE_AUTH;

    fbData.setResponseSize(4096);

    Firebase.begin(&config, &auth);

    Firebase.reconnectWiFi(true);

    Serial.println("Firebase Iniciado. Conectando...");

    while(1) {

        if (WiFi.status() == WL_CONNECTED && Firebase.ready()) {

```

```

        // 1. PREPARAR DATOS

        FirebaseJson json;

        float t_local, h_local, h_sp_local, f_sp_local,
h_out_local, f_out_local;

        xSemaphoreTake(xDataMutex, portMAX_DELAY);

        t_local = sharedTemp; h_local = sharedHum;

        h_sp_local = heater.setpoint; h_out_local = heater.output;
        f_sp_local = fan.setpoint; f_out_local = fan.output;

        xSemaphoreGive(xDataMutex);

        json.set("temperatura", t_local);

        json.set("humedad", h_local);

        json.set("heater_sp", h_sp_local);

        json.set("heater_out", h_out_local);

        json.set("fan_sp", f_sp_local);

        json.set("fan_out", f_out_local);

        // 2. ENVIAR DATOS (ESTO SE MANTIENE PARA MONITOREO)

        Serial.print("Enviando a Firebase... ");

        if (Firebase.updateNode(fbData, "/secadora/estado", json))
{
            Serial.println("OK!");
        } else {

            Serial.print("ERROR: ");

            Serial.println(fbData.errorReason());

```



```

    }

    }

    vTaskDelay(pdMS_TO_TICKS(2000));
}

}

/*=====SETUP Y LOOP =====*/
void setup() {
    Serial.begin(115200);

    Wire.begin();

    Serial.print("Conectando WiFi a: ");
    Serial.println(WIFI_SSID);
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    while(WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }

    Serial.println("\nWiFi Conectado!");

    Serial.print("IP: ");

    Serial.println(WiFi.localIP());
}

```

```
xDataMutex = xSemaphoreCreateMutex();

xTaskCreatePinnedToCore(TaskSensors, "Sensors", 4096, NULL, 2,
NULL, 1);

xTaskCreatePinnedToCore(TaskControl, "Control", 4096, NULL, 3,
NULL, 1);

xTaskCreatePinnedToCore(TaskUI, "UI", 4096, NULL, 1,
NULL, 1);

xTaskCreatePinnedToCore(TaskFirebase, "Firebase", 8192, NULL, 1,
NULL, 0);
}

void loop() {

    vTaskDelete(NULL);
}
```